

```
```bash echo -e "Plants create [MASK]
through a process known as
photosynthesis." | transformers run --
task fill-mask --model google-
bert/bert-base-uncased --device 0 ```
```

[https://huggingface.co/docs/transformers/main/en/model\\_doc/bert#transformers](https://huggingface.co/docs/transformers/main/en/model_doc/bert#transformers)

## Detailed Documentation

---

```
class <span class="code-
inline">transformers.BertConfigtransformers.BertConfighttps</sp
```

---

class

```
transformers.BertConfigtransformers.BertConfighttps ://github.com/huggir
"vocab_size", "val": " = 30522"}, {"name": "hidden_size", "val": " = 768"},
{"name": "num_hidden_layers", "val": " = 12"}, {"name": "num_attention_heads",
"val": " = 12"}, {"name": "intermediate_size", "val": " = 3072"}, {"name":
"hidden_act", "val": " = 'gelu'"}, {"name": "hidden_dropout_prob", "val": " = 0.1"},
{"name": "attention_probs_dropout_prob", "val": " = 0.1"}, {"name":
"max_position_embeddings", "val": " = 512"}, {"name": "type_vocab_size", "val":
" = 2"}, {"name": "initializer_range", "val": " = 0.02"}, {"name": "layer_norm_eps",
```

```
"val": " = 1e-12"}, {"name": "pad_token_id", "val": " = 0"}, {"name": "use_cache",
"val": " = True"}, {"name": "classifier_dropout", "val": " = None"}, {"name":
"**kwargs", "val": ""}] - **vocab_size** (int, *optional*, defaults to 30522) --
```

Vocabulary size of the BERT model. Defines the number of different tokens that can be represented by the

`inputs_ids` passed when calling `[BertModel]`  
(`/docs/transformers/main/en/model_doc/bert#transformers.BertModel`) or  
`TFBertModel`.

- `**hidden_size**` ( `int`, \*optional\*, defaults to 768) --

Dimensionality of the encoder layers and the pooler layer.

- `**num_hidden_layers**` ( `int`, \*optional\*, defaults to 12) --

Number of hidden layers in the Transformer encoder.

- `**num_attention_heads**` ( `int`, \*optional\*, defaults to 12) --

Number of attention heads for each attention layer in the Transformer encoder.

- `**intermediate_size**` ( `int`, \*optional\*, defaults to 3072) --

---

`class` `<span class="code-`

`inline">transformers.BertTokenizertransformers.BertTokenizerhtt`

---

`class`

```
transformers.BertTokenizertransformers.BertTokenizerhttps://github.com,
"vocab_file", "val": ""}, {"name": "do_lower_case", "val": " = True"}, {"name":
"do_basic_tokenize", "val": " = True"}, {"name": "never_split", "val": " = None"},
{"name": "unk_token", "val": " = '[UNK]'}}, {"name": "sep_token", "val": " =
'[SEP]'}}, {"name": "pad_token", "val": " = '[PAD]'}}, {"name": "cls_token", "val": "
= '[CLS]'}}, {"name": "mask_token", "val": " = '[MASK]'}}, {"name":
"tokenize_chinese_chars", "val": " = True"}, {"name": "strip_accents", "val": " =
None"}, {"name": "clean_up_tokenization_spaces", "val": " = True"}, {"name":
"**kwargs", "val": ""}] - **vocab_file** (str) --
```

File containing the vocabulary.

- **do\_lower\_case** ( bool , \*optional\*, defaults to True ) --

Whether or not to lowercase the input when tokenizing.

- **do\_basic\_tokenize** ( bool , \*optional\*, defaults to True ) --

Whether or not to do basic tokenization before WordPiece.

- **never\_split** ( Iterable , \*optional\*) --

Collection of tokens which will never be split during tokenization. Only has an effect when

```
do_basic_tokenize=True
```

- **unk\_token** ( str , \*optional\*, defaults to "[UNK]" ) --

---

**build\_inputs\_with\_special\_token**transformers.BertTokenizer.buil

---

```
build_inputs_with_special_tokenstransformers.BertTokenizer.build_inputs_with_special_token_ids_0", "val": "list", {"name": "token_ids_1", "val": "typing.Optional[list[int]] = None"}]- **token_ids_0** (List[int]) --
```

List of IDs to which the special tokens will be added.

- `**token_ids_1** ( List[int] , *optional* ) --`

Optional second list of IDs for sequence pairs.0 List of [input IDs] ([../glossary#input-ids](#)) with the appropriate special tokens.

Build model inputs from a sequence or a pair of sequence for sequence classification tasks by concatenating and

adding special tokens. A BERT sequence has the following format:

- single sequence: `[CLS] X [SEP]`
- pair of sequences: `[CLS] A [SEP] B [SEP]`

---

## `get_special_tokens_masktransformers.BertTokenizer.get_special`

---

```
get_special_tokens_masktransformers.BertTokenizer.get_special_tokens_maskhtoken_ids_0", "val": "list", {"name": "token_ids_1", "val": "typing.Optional[list[int]] = None"}, {"name": "already_has_special_tokens", "val": ": bool = False"}]- **token_ids_0** (List[int]) --
```

- `**token_ids_1** ( List[int] , *optional* ) --`

Optional second list of IDs for sequence pairs.

- `**already_has_special_tokens**` ( `bool`, \*optional\*, defaults to `False` ) --

Whether or not the token list is already formatted with special tokens for the model.  
`List[int]` A list of integers in the range [0, 1]: 1 for a special token, 0 for a sequence token.

Retrieve sequence ids from a token list that has no special tokens added. This method is called when adding

special tokens using the tokenizer `prepare_for_model` method.

---

## `create_token_type_ids_from_sequences``transformers.BertTokenizer`

---

`create_token_type_ids_from_sequences``transformers.BertTokenizer.create_token_type_ids_from_sequences`  
"token\_ids\_0", "val": ": list"}, {"name": "token\_ids\_1", "val": ": typing.Optional[list[int]] = None"}]- `**token_ids_0**` ( `list[int]` ) -- The first tokenized sequence.

- `**token_ids_1**` ( `list[int]`, \*optional\*) -- The second tokenized sequence.  
`list[int]` The token type ids.

Create the token type IDs corresponding to the sequences passed. [What are token type

IDs?](../glossary#token-type-ids)

Should be overridden in a subclass if the model has a special way of building

those.

---

`save_vocabulary``transformers.BertTokenizer.save_vocabulary``https://github.com/huggingface/transformers/blob/master/src/transformers/tokenization_utils.py#L1000`

---

`save_vocabulary``transformers.BertTokenizer.save_vocabulary``https://github.com/huggingface/transformers/blob/master/src/transformers/tokenization_utils.py#L1000`

```
"save_directory", "val": ":", "name": "filename_prefix", "val": ":", "name": "vocab_file", "val": ":", "name": "do_lower_case", "val": ":", "name": "unk_token", "val": ":", "name": "sep_token", "val": ":", "name": "pad_token", "val": ":", "name": "cls_token", "val": ":", "name": "mask_token", "val": ":", "name": "tokenize_chinese_chars", "val": ":", "name": "strip_accents", "val": ":", "name": "**kwargs", "val": ":"], typing.Optional[str] = None"]]
```

---

`class` `<span class="code-`

`inline">transformers.BertTokenizerFast``transformers.BertTokenizerFast`

---

`class`

```
transformers.BertTokenizerFasttransformers.BertTokenizerFasthttps://github.com/huggingface/transformers/blob/master/src/transformers/tokenization_utils.py#L1000
"vocab_file", "val": " = None"}, {"name": "tokenizer_file", "val": " = None"},
{"name": "do_lower_case", "val": " = True"}, {"name": "unk_token", "val": " = '[UNK]'",
" = '[UNK]'", {"name": "sep_token", "val": " = '[SEP]'", {"name": "pad_token", "val":
" = '[PAD]'", {"name": "cls_token", "val": " = '[CLS]'", {"name": "mask_token",
"val": " = '[MASK]'", {"name": "tokenize_chinese_chars", "val": " = True"},
{"name": "strip_accents", "val": " = None"}, {"name": "**kwargs", "val": ""}]-
vocab_file (str) --
```

File containing the vocabulary.

- `**do_lower_case**` ( `bool` , \*optional\*, defaults to `True` ) --

Whether or not to lowercase the input when tokenizing.

- `**unk_token**` ( `str`, \*optional\*, defaults to `"[UNK]"` ) --

The unknown token. A token that is not in the vocabulary cannot be converted to an ID and is set to be this

- `**sep_token**` ( `str`, \*optional\*, defaults to `"[SEP]"` ) --

The separator token, which is used when building a sequence from multiple sequences, e.g. two sequences for

sequence classification or for a text and a question for question answering. It is also used as the last

token of a sequence built with special tokens.

---

## build\_inputs\_with\_special\_tokenstransformers.BertTokenizerFast

---

build\_inputs\_with\_special\_tokenstransformers.BertTokenizerFast.build\_inputs\_wit  
 "token\_ids\_0", "val": "", {"name": "token\_ids\_1", "val": " = None"}]-  
 \*\*token\_ids\_0\*\* ( `List[int]` ) --

List of IDs to which the special tokens will be added.

- `**token_ids_1**` ( `List[int]`, \*optional\*) --

Optional second list of IDs for sequence pairs.0 `List[int]` List of [input IDs]  
 (../glossary#input-ids) with the appropriate special tokens.

Build model inputs from a sequence or a pair of sequence for sequence classification tasks by concatenating and

adding special tokens. A BERT sequence has the following format:

- single sequence: `[CLS] X [SEP]`
- pair of sequences: `[CLS] A [SEP] B [SEP]`

---

`class`

`transformers.BertModel`

---

`class`

```
transformers.BertModel ://github.com/huggingf
"config", "val": ""}, {"name": "add_pooling_layer", "val": " = True"}]- **config**
([BertModel]
(/docs/transformers/main/en/model_doc/bert# transformers.BertModel)) --
```

Model configuration class with all the parameters of the model. Initializing with a config file does not

load the weights associated with the model, only the configuration. Check out the

```
[from_pretrained ()]
```

```
(/docs/transformers/main/en/main_classes/model# transformers.PreTrainedModel)
method to load the model weights.
```

- `**add_pooling_layer**` ( `bool` , \*optional\*, defaults to `True` ) --

Whether to add a pooling layer



The model can behave as an `encoder` (with only self-attention) as well as a decoder, in which case a layer of

cross-attention is added between the self-attention layers, following the architecture described in [Attention is

all you need](https://huggingface.co/papers/1706.03762) by Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit,

Llion Jones, Aidan N. Gomez, Lukasz Kaiser and Illia Polosukhin.

---

**forwardtransformers.BertModel.forward**<https://github.com/huggingface/transformers>

---

```
forwardtransformers.BertModel.forwardhttps://github.com/huggingface/transformers
"input_ids", "val": ": typing.Optional[torch.Tensor] = None"}, {"name":
"attention_mask", "val": ": typing.Optional[torch.Tensor] = None"}, {"name":
"token_type_ids", "val": ": typing.Optional[torch.Tensor] = None"}, {"name":
"position_ids", "val": ": typing.Optional[torch.Tensor] = None"}, {"name":
"inputs_embeds", "val": ": typing.Optional[torch.Tensor] = None"}, {"name":
"encoder_hidden_states", "val": ": typing.Optional[torch.Tensor] = None"},
{"name": "encoder_attention_mask", "val": ": typing.Optional[torch.Tensor] =
None"}, {"name": "past_key_values", "val": ":
typing.Optional[transformers.cache_utils.Cache] = None"}, {"name":
"use_cache", "val": ": typing.Optional[bool] = None"}, {"name": "cache_position",
"val": ": typing.Optional[torch.Tensor] = None"}, {"name": "**kwargs", "val": ":
typing_extensions.Unpack[transformers.utils.generic.TransformersKwargs]
input_ids (torch.Tensor of shape (batch_size, sequence_length) ,
optional) --
```

Indices of input sequence tokens in the vocabulary. Padding will be ignored by default.

Indices can be obtained using `[AutoTokenizer]` ([/docs/transformers/main/en/model\\_doc/auto#transformers.AutoTokenizer](/docs/transformers/main/en/model_doc/auto#transformers.AutoTokenizer)).

See `[PreTrainedTokenizer.encode()]` ([/docs/transformers/main/en/internal/tokenization\\_utils#transformers.PreTrainedTokenizer.encode](/docs/transformers/main/en/internal/tokenization_utils#transformers.PreTrainedTokenizer.encode)) and

`[PreTrainedTokenizer.__call__()]` ([/docs/transformers/main/en/internal/tokenization\\_utils#transformers.PreTrainedTokenizer.\\_\\_call\\_\\_](/docs/transformers/main/en/internal/tokenization_utils#transformers.PreTrainedTokenizer.__call__)) for details.

[What are input IDs?](../glossary#input-ids)

- **attention\_mask** (`torch.Tensor` of shape `(batch_size, sequence_length)`, \*optional\*) --

Mask to avoid performing attention on padding token indices. Mask values selecte